Section 6.3: Distributed Transaction Orchestration & Idempotency

Part of a 3-File Distributed Systems Series (File 3 of 3)

> ***Study Note:*** *This section bridges theory and production engineering. Extra weight is given to failure scenario walkthroughs, idempotency proofs, and design trade-offs between orchestration approaches. Every pattern includes a concrete comparison of when to prefer it over its alternatives.*

---

1. Atomic Commit Protocols

Why Distributed Transactions Are Hard — The Dual-Write Problem

Before any protocol, understand the root problem:

```
Scenario: Service S needs to:
  1. Write to Database D
  2. Publish event to Message Queue MQ

Unsafe execution:
  S writes to D ✓
  S crashes
  MQ publish never happens → System is inconsistent

Or:
  S writes to D ✓
  MQ publish ✓
  D crashes before commit → DB rolled back, but event already sent → Ghost event
```

There is no atomic write across two independent systems without a protocol. This is why 2PC, Sagas, and the Outbox pattern exist.

1.1 Two-Phase Commit (2PC)

Protocol Flow:

```
Participants: Coordinator C, Participants P1, P2, P3

Phase 1 — Prepare:
  C → P1, P2, P3: "PREPARE transaction T"
  Each Pi:
    - Acquires locks on affected rows
    - Writes to WAL (write-ahead log) — prepared but not committed
    - Replies: VOTE_COMMIT or VOTE_ABORT

Phase 2a — Commit (if all voted commit):
  C → P1, P2, P3: "COMMIT T"
  Each Pi: releases locks, commits WAL changes, replies ACK

Phase 2b — Abort (if any voted abort):
  C → P1, P2, P3: "ABORT T"
  Each Pi: releases locks, rolls back WAL changes
```

Failure Analysis (Proof of Vulnerability):

```
Timeline of coordinator crash vulnerability:

T=0:  C sends PREPARE to P1, P2, P3
T=1:  P1, P2, P3 all reply VOTE_COMMIT (holding locks)
T=2:  C decides to COMMIT
T=3:  C sends COMMIT to P1 only
T=4:  C CRASHES

State:
  P1: Committed ✓ (received COMMIT)
  P2: BLOCKED — holds locks, waiting for COMMIT or ABORT
  P3: BLOCKED — holds locks, waiting for COMMIT or ABORT
  C:  Dead

P2 and P3 CANNOT:
  - Commit unilaterally (might conflict with C's decision to abort)
  - Abort unilaterally (P1 already committed — would violate atomicity)
  - Proceed with anything — locks held indefinitely

Resolution: Wait for C to recover (may take minutes/hours)
This is the "blocking" problem of 2PC.
```

When 2PC is still the right choice:

Scenario	Use 2PC?	Reason
Same database engine, multiple shards	✓ Yes	DB-native 2PC (XA) handles recovery transparently
Short-lived OLTP transactions (<100ms)	✓ Yes	Coordinator crash window is tiny

Scenario	Use 2PC?	Reason
Cross-service with different DBs	✗ No	Blocking risk too high; use Saga
Long-running workflow (seconds-minutes)	✗ No	Locks held too long; deadlock risk

---

1.2 Three-Phase Commit (3PC)

3PC adds a “pre-commit” phase to eliminate the coordinator crash blocking problem.

```
Phase 1: CanCommit
  C → all: "Can you commit T?"
  Pi replies YES/NO
  No locks yet, no state change — just a feasibility check

Phase 2: PreCommit
  If all YES: C → all: "PREPARE to commit"
  Pi: Acquires locks, writes to WAL, replies ACK
  If any NO: C → all: "ABORT"

Phase 3: Commit
  C → all: "COMMIT"
  Pi: Commit and release locks

If C crashes after Phase 2 (PreCommit ACKed by all):
  Participants can safely commit after timeout
  (Any participant that got PreCommit knows all others did too)
```

Why 3PC is still imperfect (proof):

3PC assumes a synchronous network (messages eventually delivered within timeout). Under a network partition:

```
Partition scenario:
  P1 receives PreCommit from C
  Partition occurs — P2 never receives PreCommit
  C crashes during partition

  P1's timeout fires: P1 decides to COMMIT (saw PreCommit)
  P2's timeout fires: P2 decides to ABORT (never saw PreCommit)

  Result: P1 committed, P2 aborted → Split-brain inconsistency
```

Conclusion: 3PC solves coordinator crash blocking but introduces network partition split-brain. In practice, 2PC + careful coordinator HA (fast recovery) outperforms 3PC in real-world distributed systems.

1.3 2PC vs. 3PC vs. Saga — Full Comparison

Dimension	2PC	3PC	Saga
Atomicity guarantee	Strong (ACID)	Strong (sync network only)	Eventual (compensating)
Blocking on coordinator crash	Yes	No	No
Partition tolerance	Poor	Poor	Good
Lock duration	Held until Phase 2	Held from Phase 2	No global locks
Latency	2 round trips	3 round trips	1 step at a time
Failure handling	Rollback entire txn	Rollback with timeout	Compensating transactions
Best for	Same-DB cross-shard	Rarely used in practice	Cross-service workflows
Production examples	PostgreSQL XA, MySQL XA	Rarely deployed	Stripe, Uber, Netflix

---

2. Saga Patterns

A Saga is a sequence of local transactions. Each step either succeeds and triggers the next, or fails and triggers compensating (undo) transactions for all prior steps.

Key property: Sagas provide ACD (Atomic, Consistent, Durable) but **not Isolated**. Between steps, intermediate state is visible.

2.1 Choreography Pattern

Event-driven, decentralized coordination:

```
No central coordinator. Each service publishes events that trigger the next
service.
```

```
Order Creation Saga (choreography):

  OrderService           InventoryService      PaymentService
      |                        |                     |
      |── creates Order ──────►|                     |
      |   publishes OrderCreated                     |
      |                        |                     |
      |              reserves inventory              |
      |              publishes InventoryReserved ──►|
      |                        |                     |
      |                        |           charges payment
      |                        |           publishes PaymentProcessed
      |◄──────────────────────────────────────────-─┤
      |                        |                     |
  confirms order               |                     |

Failure — PaymentFailed event:
      |◄──────────────── PaymentFailed ──────────────┤
      |                        |                     |
      |              publishes InventoryRelease      |
      |                        |                     |
      |         releases reservation                 |
      |                        |                     |
  cancels order                |                     |
```

Pros of choreography: - Fully decoupled — no service knows about others - Resilient — no single point of failure - Easy to add new steps (subscribe to existing events)

Cons: - Hard to track overall saga state (distributed across services) - Debugging failures requires correlating events across multiple logs - Cyclic dependencies possible if not careful

2.2 Orchestration Pattern

Central saga orchestrator maintains state machine:

```
class OrderSagaOrchestrator:
    """State machine persisted in durable store (DB/Redis)."""

    STEPS = [
        ("reserve_inventory", "release_inventory"),   # (action, compensation)
        ("charge_payment",    "refund_payment"),
        ("confirm_order",     "cancel_order"),
    ]

    def execute(self, order_id: str):
        saga_state = self.load_or_create(order_id)

        for step_idx, (action, compensation) in enumerate(self.STEPS):
            if step_idx < saga_state.completed_steps:
                continue  # Already done (idempotent resume)
```

```
            try:
                result = self.call_service(action, order_id)
                saga_state.completed_steps = step_idx + 1
                saga_state.step_results[action] = result
                self.save(saga_state)  # Persist progress

            except Exception as e:
                logger.error(f"Step {action} failed: {e}")
                self.compensate(order_id, saga_state, step_idx)
                raise SagaFailedError(f"Saga aborted at step {action}")

    def compensate(self, order_id: str, state: SagaState, failed_at: int):
        """Run compensating transactions in reverse order."""
        for step_idx in range(failed_at - 1, -1, -1):
            _, compensation = self.STEPS[step_idx]
            try:
                self.call_service(compensation, order_id)
                state.compensated_steps.add(step_idx)
                self.save(state)
            except Exception as e:
                # Compensation failure → escalate to dead letter / human review
                self.escalate(order_id, compensation, e)
```

Pros of orchestration: - Single place to see saga state and progress - Easier to debug — one orchestrator log tells the story - Natural place for retry logic, timeouts, escalations

Cons: - Orchestrator is a central point (mitigate with HA + idempotency) - Orchestrator knows about all participants (coupling)

2.3 Choreography vs. Orchestration — When to Use Which

Scenario	Recommendation
<4 services involved	Choreography (simple, decoupled)
Complex state machine with branching	Orchestration (visibility, control)
Team autonomy is critical	Choreography (services own their events)
Need audit trail of saga progress	Orchestration (single state store)
Long-running (hours/days)	Orchestration (timeout management centralized)
New saga steps added frequently	Choreography (add subscriber, no orchestrator change)

2.4 Compensation Design — Best Effort vs. Guaranteed

```
Not all compensating actions are equal:

GUARANTEED compensations (must succeed):
  - Refund payment (money movement)
  - Release database locks
  Implementation: Retry indefinitely with exponential backoff
                  Dead letter queue after N retries → human escalation

BEST EFFORT compensations (may be impossible):
  - Cancel shipment that already left warehouse
  - Un-send email notification
  Implementation: Attempt once, log failure, alert operator
                  Accept "already shipped" as valid terminal state

Design rule: Write compensating transactions assuming the forward
transaction fully committed. Never assume partial state.
```

---

3. Transactional Outbox Pattern

The Dual-Write Problem Revisited

Why you cannot safely do this:

```
# UNSAFE — dual write
def create_order(user_id, items):
    db.execute("INSERT INTO orders ...", ...)     # Step 1
    message_queue.publish("OrderCreated", ...)    # Step 2
    # If crash between step 1 and 2: order exists, no event sent
    # If step 2 fails: order exists, inventory never reserved
```

Safe Implementation: Outbox Table

Core idea: Write the event to a database table *in the same ACID transaction* as the business data. A separate relay process reads the outbox and publishes to the message queue.

```
-- Schema
CREATE TABLE orders (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL,
    status TEXT NOT NULL,
    total_cents INT NOT NULL,
    created_at TIMESTAMP DEFAULT now()
);
```

```
CREATE TABLE outbox (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    aggregate_type TEXT NOT NULL,   -- e.g., "Order"
    aggregate_id UUID NOT NULL,     -- e.g., order ID
    event_type TEXT NOT NULL,       -- e.g., "OrderCreated"
    payload JSONB NOT NULL,
    created_at TIMESTAMP DEFAULT now(),
    dispatched_at TIMESTAMP         -- NULL = pending, non-NULL = sent
);
```

```
def create_order(user_id: str, items: list) -> Order:
    order = Order(id=uuid7(), user_id=user_id, items=items)

    # SINGLE ATOMIC TRANSACTION — both succeed or both fail
    with db.transaction():
        db.execute("""
            INSERT INTO orders (id, user_id, status, total_cents)
            VALUES (%s, %s, 'pending', %s)
        """, [order.id, user_id, order.total_cents])

        db.execute("""
            INSERT INTO outbox (aggregate_type, aggregate_id, event_type, payload)
            VALUES ('Order', %s, 'OrderCreated', %s)
        """, [order.id, json.dumps(order.to_dict())])

    # Transaction committed → both rows exist atomically
    return order
```

Outbox Relay Process:

```
def relay_outbox():
    """Runs continuously; can be deployed as separate microservice."""
    while True:
        # Fetch pending events (batch for efficiency)
        events = db.query("""
            SELECT * FROM outbox
            WHERE dispatched_at IS NULL
            ORDER BY created_at
            LIMIT 100
            FOR UPDATE SKIP LOCKED  -- Prevents multiple relay instances from
racing
        """)

        for event in events:
            try:
                message_queue.publish(
                    topic=event.event_type,
                    key=str(event.aggregate_id),   # Partition key for ordering
                    value=event.payload
                )
                db.execute("""
```

```
                    UPDATE outbox SET dispatched_at = now() WHERE id = %s
                """, [event.id])
            except Exception as e:
                logger.warning(f"Failed to dispatch event {event.id}: {e}")
                # Will retry on next poll cycle

        time.sleep(0.1)  # Poll every 100ms
```

Alternative: CDC (Change Data Capture) instead of polling:

```
Debezium reads PostgreSQL WAL (write-ahead log) directly:
  - No polling overhead
  - Sub-second latency (vs. polling interval)
  - No FOR UPDATE SKIP LOCKED needed
  - Works even if relay process is completely separate from app

Trade-off: More infrastructure (Debezium + Kafka Connect)
Use polling for: Simple setups, low event volume (<1,000/sec)
Use CDC for: High event volume, latency-sensitive, polyglot services
```

---

4. Idempotency in Distributed Systems

Why Idempotency is Non-Negotiable

In distributed systems, at-least-once delivery is the norm. Your consumer *will* see the same message twice. Your API *will* receive the same request retried. Design for it explicitly.

Formal definition: Operation f is idempotent if f(f(x)) = f(x). Applying it multiple times produces the same result as applying it once.

4.1 Idempotency Keys (Stripe Pattern)

```
def process_payment(
    amount: int,
    currency: str,
    customer_id: str,
    idempotency_key: str   # Client-generated unique key per logical request
) -> PaymentResult:

    # Step 1: Check if already processed
    existing = db.query("""
        SELECT response FROM payments WHERE idempotency_key = %s
    """, [idempotency_key])

    if existing:
        # Return IDENTICAL response as before — safe retry
```

```
        return PaymentResult(**json.loads(existing.response))

    # Step 2: Process (this is the only-once execution)
    try:
        charge = stripe.charge.create(
            amount=amount,
            currency=currency,
            customer=customer_id,
            idempotency_key=idempotency_key  # Pass to Stripe too!
        )
        result = PaymentResult(id=charge.id, status="succeeded")
    except stripe.error.CardError as e:
        result = PaymentResult(id=None, status="failed", error=str(e))

    # Step 3: Store with unique constraint on idempotency_key
    db.execute("""
        INSERT INTO payments (idempotency_key, amount, currency, status, response)
        VALUES (%s, %s, %s, %s, %s)
        ON CONFLICT (idempotency_key) DO NOTHING
    """, [idempotency_key, amount, currency, result.status,
json.dumps(result.to_dict())])
    # ON CONFLICT handles race condition if two requests arrive simultaneously

    return result
```

TTL consideration:

```
Idempotency keys should expire (not kept forever):
  Stripe: 24 hours
  Typical e-commerce: 7 days (order processing window)
  Financial settlement: 30 days (dispute window)

After TTL: Same key treated as new request (client should use new key anyway)
Cleanup: Scheduled job DELETE FROM payments WHERE created_at < now() - interval
'30 days'
```

4.2 Exactly-Once Semantics — Layered Approach

```
Exactly-once = at-least-once delivery + idempotent consumer

Layer 1: Message queue delivers at-least-once (Kafka, SQS guarantee this)
Layer 2: Consumer deduplicates using event ID

Consumer deduplication:
def handle_order_created(event: dict):
    event_id = event["id"]

    # Redis SETNX (set if not exists) with TTL
    was_set = redis.set(
        f"processed_event:{event_id}",
        "1",
```

```
        nx=True,    # Only set if key doesn't exist
        ex=86400    # Expire after 24 hours
    )

    if not was_set:
        logger.info(f"Duplicate event {event_id} — skipping")
        return  # Idempotent: skip without error

    # Process exactly once
    reserve_inventory(event["order_id"], event["items"])
```

Idempotency at the Stripe integration layer (worked example):

```
Scenario: Your server charges Stripe, but network times out before response.
          You don't know if Stripe charged the card or not.
          Retry?

Without idempotency key: Stripe receives two charge requests → double charge
With idempotency key:    Stripe sees same key → returns same response, no double
charge

Implementation:
  client_key = f"charge_{order_id}_{attempt_number}"
  # attempt_number increments only if client KNOWS the previous attempt failed
  # If timeout (unknown): reuse same key → safe retry

  stripe.Charge.create(
      amount=1000,
      idempotency_key=client_key
  )
```

4.3 Idempotent vs. Non-Idempotent Operations

Operation	Idempotent?	Why / Fix
`PUT /users/123 {name: "Alice"}`	✓ Yes	Full replacement — same result every time
`DELETE /orders/456`	✓ Yes	Already deleted = not found = same outcome
`POST /orders`	✗ No	Creates new order each call
`POST /orders` + idempotency key	✓ Yes	Key prevents duplicate creation
`PATCH /account {balance: +100}`	✗ No	Relative update — adds 100 each call

Operation	Idempotent?	Why / Fix
`PATCH /account {balance: 500}`	✓ Yes	Absolute set — idempotent
`GET /resource`	✓ Yes	Read-only, always idempotent

---

5. Interview Problems with Full Solutions

Problem 1: Exactly-Once Payment Processing with Retries

Requirements: - Payment API called by mobile client - Mobile may retry on network timeout (doesn't know if first call succeeded) - Must never double-charge - Must handle Stripe API being unreachable

Architecture:

```
Mobile Client → Payment API → [Idempotency Check] → Stripe → [Outbox] →
Notification Service

Key design decisions:
1. Client generates idempotency key before first attempt
2. API persists key + result atomically before returning
3. Retry: API returns cached result (no Stripe call)
4. Outbox ensures notifications regardless of API crash
```

Full implementation:

```
class PaymentService:
    def process_payment(self, request: PaymentRequest) -> PaymentResult:
        key = request.idempotency_key

        # === IDEMPOTENCY CHECK ===
        existing = db.query(
            "SELECT * FROM payments WHERE idempotency_key = %s", [key]
        )
        if existing:
            return PaymentResult.from_db(existing)  # Safe return

        # === PROCESS PAYMENT ===
        try:
            stripe_result = stripe.Charge.create(
                amount=request.amount_cents,
                currency=request.currency,
                customer=request.customer_id,
                idempotency_key=key   # Stripe-level dedup
            )
```

```
            status = "succeeded"
            stripe_charge_id = stripe_result.id
            error_message = None

        except stripe.error.CardError as e:
            # Card declined — deterministic failure, don't retry
            status = "failed"
            stripe_charge_id = None
            error_message = str(e)

        except stripe.error.APIConnectionError:
            # Network error — Stripe may or may not have charged
            # Query Stripe for actual status before giving up
            actual = self._query_stripe_status(key)
            if actual:
                status = actual.status
                stripe_charge_id = actual.id
            else:
                raise RetryableError("Stripe unreachable — client should retry
with same key")

        # === ATOMIC PERSIST (order + outbox) ===
        with db.transaction():
            db.execute("""
                INSERT INTO payments
                  (idempotency_key, amount_cents, currency, status,
stripe_charge_id, error_message)
                VALUES (%s, %s, %s, %s, %s, %s)
                ON CONFLICT (idempotency_key) DO NOTHING
            """, [key, request.amount_cents, request.currency, status,
stripe_charge_id, error_message])

            if status == "succeeded":
                db.execute("""
                    INSERT INTO outbox (aggregate_type, aggregate_id, event_type,
payload)
                    VALUES ('Payment', %s, 'PaymentSucceeded', %s)
                """, [stripe_charge_id, json.dumps({"key": key, "amount":
request.amount_cents})])

        return PaymentResult(status=status, charge_id=stripe_charge_id)

    def _query_stripe_status(self, idempotency_key: str):
        """Check if Stripe processed our request despite network failure."""
        try:
            charges = stripe.Charge.list(limit=1)
            for charge in charges.auto_paging_iter():
                if charge.idempotency_key == idempotency_key:
                    return charge
        except Exception:
            return None
```

Failure scenario walkthrough:

```
T=0:  Client sends payment request, key=K1
T=1:  API calls Stripe: ACCEPTED (charge_id=ch_abc)
T=2:  Network drops — client never receives response
T=3:  Client retries with SAME key K1

Server at T=3:
  existing = db.query(key=K1) → FOUND (from T=1 insertion)
  Returns: {status: "succeeded", charge_id: "ch_abc"}
  No Stripe call made → No double charge ✓
```

Problem 2: Order Processing Saga with Compensation

Requirements: - E-commerce order: reserve inventory → charge payment → ship - If any step fails, fully reverse prior steps - Compensation failures must not leave system in unknown state

Saga with compensation tracking:

```
from enum import Enum

class SagaStatus(Enum):
    PENDING = "pending"
    EXECUTING = "executing"
    COMPENSATING = "compensating"
    COMPLETED = "completed"
    FAILED = "failed"
    ESCALATED = "escalated"

class OrderSaga:
    STEPS = [
        {
            "name": "reserve_inventory",
            "service": InventoryService,
            "action": "reserve",
            "compensation": "release",
            "compensation_type": "guaranteed"  # Must always succeed
        },
        {
            "name": "charge_payment",
            "service": PaymentService,
            "action": "charge",
            "compensation": "refund",
            "compensation_type": "guaranteed"
        },
        {
            "name": "initiate_shipment",
            "service": ShipmentService,
            "action": "create",
            "compensation": "cancel",
            "compensation_type": "best_effort"  # May have already shipped
```

```
        }
    ]

    def execute(self, order_id: str):
        saga = self.load_or_create(order_id)

        # Forward execution
        for i, step in enumerate(self.STEPS):
            if i < len(saga.completed):
                continue  # Idempotent resume after crash

            try:
                result = step["service"].call(step["action"], order_id)
                saga.completed.append({"step": step["name"], "result": result})
                self.persist(saga)

            except Exception as e:
                saga.status = SagaStatus.COMPENSATING
                self.persist(saga)
                self.compensate(order_id, saga, failed_at=i, reason=str(e))
                return

        saga.status = SagaStatus.COMPLETED
        self.persist(saga)

    def compensate(self, order_id: str, saga, failed_at: int, reason: str):
        for i in range(failed_at - 1, -1, -1):
            step = self.STEPS[i]
            result = saga.completed[i]["result"]

            try:
                step["service"].call(step["compensation"], order_id, result)
                saga.compensated.append(step["name"])
                self.persist(saga)

            except Exception as comp_err:
                if step["compensation_type"] == "guaranteed":
                    # Retry indefinitely via dead letter queue
                    self.dead_letter_queue.publish({
                        "order_id": order_id,
                        "step": step["name"],
                        "compensation": step["compensation"],
                        "error": str(comp_err),
                        "requires_human_review": True
                    })
                    saga.status = SagaStatus.ESCALATED
                    self.persist(saga)
                    alert_on_call(f"Compensation failed for {order_id}:
{comp_err}")
                    return
                else:
                    # best_effort: log and continue
                    logger.warning(f"Best-effort compensation {step['name']}
failed: {comp_err}. Continuing.")
```

```
        saga.status = SagaStatus.FAILED
        self.persist(saga)
```

Intermediate state visibility (the isolation problem):

```
Between step 1 (inventory reserved) and step 3 (shipment confirmed):
  Inventory shows "reserved" — not yet shipped
  Payment shows "charged" — money taken

A concurrent read query during this window sees inconsistent state.
This is unavoidable with Sagas (no global isolation).

Mitigations:
  1. Semantic locks: Mark order as "processing" — UI shows "Order being processed"
  2. Aggregate design: Don't expose individual service state; expose saga-level
state
  3. Timeout: If saga doesn't complete in 5 minutes → compensate automatically
```

Problem 3: Transactional Outbox for Multi-Region Event Fan-Out

Requirements: - Events generated in US-East must reach consumers in EU and APAC - Must survive US-East database failover (RDS promoted replica) - At-least-once delivery with idempotent consumers - Ordering: events per aggregate (per order) must be in order

Architecture:

```
US-East DB (Primary)
  ├── orders table
  └── outbox table
        ↓ (CDC via Debezium reads WAL)
  Kafka (US-East)
  ├── Topic: order-events (partitioned by order_id)
        ↓ (Kafka MirrorMaker replication)
  Kafka (EU-West)
  Kafka (APAC)
        ↓ (consumers in each region)
  EU Notification Service
  APAC Analytics Service
```

Ordering guarantee:

```
# Outbox relay publishes with partition key = aggregate_id
message_queue.publish(
    topic="order-events",
    key=str(event.aggregate_id),   # Same order → same Kafka partition
    value=json.dumps(event.payload),
    headers={"event_id": str(event.id), "sequence": str(event.sequence_num)}
```

```
)

# Kafka guarantees: same partition key → same partition → ordered delivery
# Per-order ordering maintained ✓
# Cross-order ordering NOT guaranteed (different partitions)
```

Consumer idempotency across regions:

```
def handle_order_event(event: dict, region: str):
    event_id = event["event_id"]
    order_id = event["aggregate_id"]

    # Region-scoped dedup: same event, different region, independent processing
    dedup_key = f"{region}:processed:{event_id}"
    was_new = redis.set(dedup_key, "1", nx=True, ex=7 * 86400)  # 7-day TTL
    if not was_new:
        return  # Already processed in this region

    if region == "eu":
        eu_notification_service.send(order_id, event["event_type"])
    elif region == "apac":
        apac_analytics_service.record(order_id, event)
```

Failover scenario:

```
US-East primary DB fails at T=0
RDS promotes replica at T=30s

Outbox state at T=0:
  Row A: dispatched_at = not null (already sent)
  Row B: dispatched_at = null (pending — Debezium hadn't read it yet)

After failover:
  Row B exists on promoted replica (WAL replication was current)
  Debezium reconnects to new primary
  Reads Row B from outbox → publishes to Kafka
  Consumers see Row B, process it

Row A: May be re-read if Debezium's offset was before Row A dispatch
  Consumer dedup key handles this → skipped ✓

Net result: At-least-once delivery preserved across failover. ✓
```

Quick Reference: Transactions & Idempotency Cheat Sheet

```
Protocol selection:
  Same DB, cross-shard          → 2PC (DB-native XA)
```

```
  Short txn, low failure risk    → 2PC
  Cross-service, >2 participants → Saga
  Long-running workflow          → Saga (Orchestration)
  Simple, decoupled services     → Saga (Choreography)

Outbox pattern:
  Problem: Dual-write (DB + MQ) is unsafe
  Solution: Write event to DB table in same transaction → relay to MQ
  Options: Polling relay (simple) vs CDC/Debezium (low latency)
  Ordering: Use aggregate_id as MQ partition key

Idempotency:
  Client generates key → same key = same request
  Server: check key → if exists, return cached response
  DB constraint: UNIQUE(idempotency_key) prevents races
  TTL: 24h–30d depending on business domain
  Pass key to all downstream calls (Stripe, external APIs)

Exactly-once = at-least-once delivery + idempotent consumer
  Consumer: check Redis SETNX before processing
  Outbox: sequence number per aggregate for ordering

Compensation types:
  Guaranteed: refund, release lock → retry indefinitely → DLQ → human
  Best-effort: cancel email, cancel shipment → attempt once → log + alert
```